COVTL Lab Console — Technical Reference

Bridge API, anchor instrumentation, clocks, caching, extension guide

Frédéric Berthommier

October 2026 — console v3 (bridge 3.0.0), engine covtl-pipeline v1.1.0

Contents

1 Architecture **1**

1.1 Module map (`js/`) 2

2 The bridge API **2**

2.1 Endpoints 3

2.2 Cross-word sandhi (covtl-sandhi) 4

2.3 The segment payload 5

3 Anchor-timing instrumentation **5**

3.1 Instrumented replay 5

3.2 Drift checks 5

3.3 Start-early and the `.polar` mapping 5

4 Clock model **6**

5 Caching **6**

6 Front-end state machine and WYSIWYG paths **6**

7 The single-file bundler **7**

8 Known environment quirks **8**

9 Extension guide **8**

1 Architecture

Three tiers, all local:

```
browser UI (ES modules, no build step)
  |  fetch JSON over http, 30 s timeout, no-cache
FastAPI bridge  (server.py, 127.0.0.1:8421)
```

```
   |  in-process import, one Pipeline per job on a thread
 covtl-pipeline  (vtl_synth)  ->  VocalTractLab 2.4 natives
```

The Python engine is the single source of truth for every phonetic decision; the browser only previews, displays and controls. When `vtl_synth` is not importable, the bridge answers `/api/status` with `backend:"mock"` and every engine-dependent endpoint returns a structured 503; the UI then runs its browser demo engine (`js/demo-engine.js`) — same interface, approximate behavior, labeled as demo.

1.1 Module map (`js/`)

`data.js`	Engine constants mirrored client-side: inventory, suggestion table, per-language examples, keyboard map, demo lexicons, polar targets.
`demo-engine.js`	Offline simulation: `demoG2P`, `demoTimeline`, `demoF0`, `demoAudio` (formant-tone synthesis + WAV encoding), `demoCompile`, `demoSegmentPayload`, `demoSynthesize`.
`render.js`	DOM renderers: token strips, guard banner, progress, result summary, downloads, examples, history; `$`, `$$`, `el`, `toast`, `log`.
`api.js`	Transport (`jfetch` with 30 s abort), the `api` facade, client-side caches, demo→live cache clearing (`clearDemoCaches`), `normalizePolar`.
`polar.js`	`PolarStage`: the continuous display — layered canvases, dual clocks, anchor stepping. Drawing policy (journey-centric): exactly ONE journey is displayed — the anchor pair under the head. Its vocalic branch is a single arc from start anchor to end anchor (full opacity); its consonantal excursion (closed loop dep → C targets → arr) is drawn jointly when the segment carries finite c samples. Everything else is eliminated — no watermark of past journeys; the tuilage is a clean switch at the anchor boundary. Reduced-vowel anchors (`@`, `6`) are hollow rings; phonetic labels cover the current journey only. The journey's origin anchor is a red dot and its target anchor a green ring, drawn above the head (stacking visibly on stationary holds), superseding the former amber "next anchor" ring.
`spectrogram.js`	`SpectrogramView`: FFT spectrogram; `setSource` accepts the anchor list for the overlay.
`compiler.js`	`SyllableCompiler` panel + `stretchLabel`/`segmentLabel`.
`app.js`	Controller: state machine, flows (preview / compile / synthesize / playback), transport wiring, keyboard, settings (`covtl.settings.v3`), history (`covtl.history.v1`).

2 The bridge API

All bodies are JSON. Errors use one structured envelope everywhere (v1-compatible `detail` plus a v2 error object):

```
{ "detail": "<message>",
  "error": { "class": "SampaGuardError", "message": "...",
             "hint": "...", "symbols": [ {symbol, position,
```

reason, suggestions}] } }

2.1 Endpoints

GET /api/status	Identity + caches: `backend` (`live|mock`), `engine:"vtl-synth"`, `version`, `bridge:"3.0.0"`, speakers, languages, defaults, cache counters (preview/compile/synth entries + hits), jobs, inventory digest, `out_root` (absolute). When live: `active` {speaker, lang} and lexicon sizes.
GET /api/health	`status` in `ok|degraded|offline`, `engine_version`, `active_speaker`, `active_lang`, `registry_warnings` (speaker probe failures, absent lexicons), `sandhi` capability block (below).
GET /api/languages	Six entries {`id, name, prosody_profile, lexicon_present`} + the active language.
GET /api/speakers?lang=	Registry {`id, label, f0_default, verified_with_lang, active`}; `warning_when_unverified` is set for non-English languages (vowels calibrated on `jd3` only); `calibration_speaker` is `jd3`.
POST /api/preview	Request {`text?, sampa?, mode: ortho|phonetic, lang, expressive, policy?, sandhi: off|minimal|standard|careful, sandhi_mode: g2p|post|pre`}. One round trip for the whole preview stack: response {`g2p_string, enriched_string, final_sampa, words[], groups[], warnings[], tokens[], rejected_symbols[], language, lexicon, inventory_digest, cache`}. Passing `sampa` re-enriches an edited G2P string; phonetic mode runs the guard only. A non-`off` `sandhi` level is applied through the *same gateways* as `Pipeline.run` (below) and degrades to `off` with a warning when Branch B is missing, the language has no rule module, or the input is a phonetic string; a bad level/mode is a 422. LRU(128); repeats answer `cache:true`.
POST /api/preview/sampa	Guard-only tokenization: {`tokens, rejected`}. (Deprecated v1 `/api/preview/g2p` still works, forwarding to the unified preview.)
POST /api/compile	Request {`text, lang, timings, guard_policy, expressive, sandhi, sandhi_mode`}. Runs the engine's anchor walk with *no synthesis*: response carries `anchors[], blocks[], nodes[], metrics`{`anchors, segments, blocks, syllables, clusters, consonants, vowels, steps, duration_s`}, `segments_preview[]` (per anchor pair: label, chronological `stretch` with the ~ coda marker, cons/vowel, kinds, n_steps), per-phoneme `targets`{`rho, theta, category`}, rejected symbols, warnings, the `sandhi` capability block. A requested sandhi level is a documented no-op here (SAMPA blocks carry no orthography — the level acts at the text→SAMPA stage) and yields a warning. Guard `error` + rejected ⇒ 422 `SampaGuardError`. LRU(64).

POST /api/synthesize	Request (superset): {`text, mode, lang, speaker, expressive, f0, timings, engine, ortho, outputMode: audio|sagittal|polar, guardPolicy, outputDir, label, exports, sandhi, sandhi_mode`}. Guard `error` is validated *before* any job starts (⇒ 422). Response {`job_id, cached, echo`}; `echo` repeats the full resolved parameter set (the log line), including `sandhi`/`sandhi_mode` and a `sandhi_note` degradation notice when Branch B is missing. The level is passed to the `Pipeline` when its signature carries it (signature-guarded like every engine option); the synth cache key includes it. Cache hit requires state `done` + all artifacts on disk.
GET /api/job/{id}	Job payload: `state` (`queued|running|cancelling|done|error|cancelled`), `stage` (`g2p|trajectory|vtl|wav|video`), `progress`, `polar_ready`, `wav_ready`, `anchors[]`, `block_count`, `duration_s`, `echo`, `result`{`sampa, warnings, duration, frames, phrases, files[], wav_url`}, error fields.
GET /api/job/{id}/polar	Segment payload (below).
POST /api/job/{id}/cancel	`queued` jobs become `cancelled` immediately; `running` jobs flip to `cancelling` (result discarded at the next checkpoint).
GET /api/job/{id}/files/{name}	`FileResponse` for any artifact registered on the job.

2.2 Cross-word sandhi (covtl-sandhi)

The bridge integrates the engine's *Branch-A hook* (`vtl_synth.utils.sandhi`, default `off`) without ever re-interpreting it. Architecture and rules live in the companion `covtl-sandhi` package (*Branch B*, six languages: fr liaison, en a→an + onset linking, it raddoppiamento + euphonic d, es enlace + y→e/o→u, pt encadeamento + EP voicing, de Glottalversatz), installed on the engine either by copying its `sandhi/` folder over the engine's stub or by pip.

- **Capability probe.** At import the bridge resolves the rule package exactly as `Pipeline` does (stub, then top-level `import sandhi`); until `SANDHI_FULL` is true the capability stays *unavailable* and `sandhi_capability()` reports {`available: false, reason: ...`}. No consumer can crash the bridge on a missing package: the preview degrades with a warning, the job echo carries a `sandhi_note`.
- **Levels** `off|minimal|standard|careful` — a monotone coverage filter over each language's rule set (minimal ⊆ standard ⊆ careful; it decides which word-pair classes may fire, never the phonology of a firing rule). **Modes** `g2p` (liaison-aware per-word gateway + pair merge, recommended), `post` (SAMPA post-processor), `pre` (orthographic rewrite before the g2p).
- **Preview path.** `/api/preview` mirrors `Pipeline.run`'s degradation order (off for phonetic input, off without Branch B, off without a rule module for the language — each with a printed warning) and then applies the same three gateways; the expressive path wraps the per-word g2p in `sandhi_oracle` and closes with `apply_sandhi_sampa` (never across a `%` breath pause). The word chips use the same per-word oracle.
- **Synthesis path.** `/api/synthesize` passes `sandhi`/`sandhi_mode` to the `Pipeline` when its signature carries them; `_resolve_sampa` applies the same gateways so the early polar

stage compiles the SAMPA the engine will actually speak (compile plan == synth plan). The cache keys (preview, compile, synth) all include the level.

- **No per-stretch link level.** The engine's articulation plan has no `link_level` attribute: a liaison consonant inserted at the text→SAMPA gateway appears in the compiled plan like any onset cluster, and the polar display follows the engine geometry unchanged (WYSIWYG contract). The nasal hiatus labels (`a .a`, `a.a` , merged `V˜.V˜` → `a .a`) are the separate V1V2 feature of the plan itself.

2.3 The segment payload

```
{ job_id, segment_count, from, rho_max, duration_s, sample_rate: 100,
  anchors:  [ { i, t, kind, label, long, breath, hold, word_end, pt } ],
  segments: [ { segment_index, t0, t1, anchor_from, anchor_to,
                label_from, label_to, blocks[...], markers{...},
                points: [ { t, rho_v, theta_v, rho_c, theta_c } ] } ] }
```

Points run at 100 Hz; null channels mark an inactive branch (the consonantal branch is null outside clusters). `theta` arrives pre-converted to the display convention and `rho` normalized over the job's radius range, so the browser never re-derives engine units.

3 Anchor-timing instrumentation

3.1 Instrumented replay

The bridge never re-implements the walk. Inside `vtl_synth.core.syltraj.build_phrase_pval`, the engine's `build_global_pval` iterates over consecutive anchor pairs; every appended block constructs a `SylBlock`, and every iteration calls `_collect_consonants` exactly once (after the possible departure plateau). `server._instrumented_phrase_build` hooks *both* — the same monkeypatch idiom as `vtl_synth.video.polar_video._recorded_build_pval`:

- a `RecBlock` subclass records construction order into a ledger;
- a wrapped `_collect_consonants` records the ledger length at each anchor-pair iteration.

Anchor k's step position is the cumulative block step count at iteration k, minus that iteration's departure plateau (a plateau at the slice tail belongs to the *next* iteration — iterations never emit plateaus after their hook). The terminal anchor is the total step count.

3.2 Drift checks

Two hard invariants raise `StructureDrift` (HTTP 500) instead of guessing: recorded blocks == engine blocks, and iteration count == `len(anchors)`−1. If the engine's walk ever changes shape, the bridge fails loudly.

3.3 Start-early and the `.polar` mapping

The synthesis worker builds the trajectory structure *first* (cheap, pure Python): `structure_compile` for anchors/blocks, then the engine's own `build_polar`/`write_polar` with the same parameters the engine will use — the `.polar` content is byte-identical to what `Pipeline.run(polar=True)`

writes. `polar_ready` flips *before* the VTL stage starts; if the early stage fails, the display waits for the engine's own export and the job carries a note.

`.polar` format v4 is JSON at 100 Hz: `trajectory.vocalic|consonantal.{rho, theta}` in *trigonometric radians* (0 East, CCW), `phonemes[]` with t_0/t_1 spans and category, `inventory[]`. The bridge converts for display when serving the segment payload:

```
theta_display = (90 - degrees(theta_trig)) mod 360   # 0 = top, clockwise
rho_norm      = rho / rho_max ,  rho_max = max(1.45, inventory rhos)
```

4 Clock model

- **Polar-only mode.** From `polar_ready` until the audio is attached, `PolarStage` advances on its own monotonic clock (performance timestamps), scaled by the speed selector.
- **Audio mastery.** Once the WAV is ready and playing, the audio element is master: the loop reads `audio.currentTime` (read-only) each tick. The handoff at block boundaries goes through `_pendingAudioSync` so the head never jumps.
- **Autoplay fallback.** If the browser refuses autoplay, the own clock keeps the display alive; the audio takes over on its `play` event (the first user gesture on *Play* unlocks it).
- **Replay.** When the audio runs to its end, the display replays from 0 on its own clock.
- **Background-tab watchdog.** `requestAnimationFrame` freezes in hidden tabs; a 50 ms interval watchdog advances the loop whenever rAF stalls (>120 ms), so start-early works even when the console tab is hidden. Rendering resumes on focus.

5 Caching

	Key	**Capacity / notes**
Preview LRU	(text, sampa, mode, lang, expressive)	128; hits answer `cache:true`
Compile LRU	(text, lang, timings JSON)	64; same `cache` flag
Synthesis	SHA-1 over the resolved parameter set *including the engine version*	32; hit requires `done` + artifacts on disk

The synthesis key *excludes* `guardPolicy` deliberately: `warn`/`off` yield identical audio, and the `error` policy is re-validated on every request (fresh or cached) so a cached job can never bypass the guard. Including the engine version invalidates entries across engine upgrades instead of serving old-engine audio.

Client side, `api.js` keeps mirrors for instant UI replay; `clearDemoCaches()` wipes them when the probe flips demo→live so simulated results never leak into live mode.

6 Front-end state machine and WYSIWYG paths

`empty`	start / after clearing input.
`preview-ready`	a preview with no rejected symbols.
`guard-warn`	rejected symbols under policy `warn`; synthesis relabeled "Synthesize anyway (N dropped)".

`guard-error`	rejected symbols under policy `error`; synthesis blocked.
`synthesizing`	job running (stage strip + progress bar).
`polar-running`	start-early: polar display alive while the WAV renders.
`ready`	job done, artifacts available.
`playing`	playback in progress.
`backend-offline`	probe failing; demo fallback offered.

A v3 rule: *late preview responses can never downgrade a running job* — `setPreviewState` only transitions from preview states (`empty`, `preview-ready`, `guard-*`, `backend-offline`). Stale responses are dropped by a sequence counter (`previewSeq`); the compiler has the same guard for in-flight compiles.

Input reset (`resetInputs`, key `R`). One action clears both source panes and the whole preview chain: drafts, `preview`, `editedSampa`, `sampaTokens`, `sampaRejected`, plus the editor widget and guard banner. It also clears every surface built on that chain: the compiler panel (`compiler.reset()` — field, tiles, chips, cards, notes, sequence bump against in-flight compiles) and the acoustic synthesis results (playback stopped, audio element detached, polar payload dropped, spectrogram cleared, scrub ticks, result summary, downloads menu; a running job is cancelled). It also bumps `previewSeq` (an in-flight preview response is dropped by the same stale-guard) and cancels the debounced auto-preview timer, so no late response can repopulate the cleared stack. The transition to `empty` is forced with `setAppState`: the no-downgrade rule in `setPreviewState` protects against *late preview responses*, not explicit user actions. Settings persist (minus the cleared sources); language, speaker, timings and history are untouched. This exists because stale chain state leaked across input modes — an *edited* SAMPA kept steering synthesis after a pane switch, and a compiled plan stayed on screen after the input was cleared (observed).

WYSIWYG synthesis. Three paths, one promise — the string you see is the string you hear:

1. *deterministic-ortho*: **S** uses the previewed final SAMPA (the same G2P callable the engine's `Pipeline.run` uses);
2. *phonetic / edited*: **E** (or phonetic mode) sends exactly the edited string (`use_g2p=False` semantics);
3. *compiler-edited*: "Use in phonetic input" pushes the compiled string to the stage, previewed, then synthesized.

7 The single-file bundler

`build_standalone.py` inlines the ES modules into `covtl-console-standalone.html` with a purely textual transform: each module becomes an IIFE factory registered in a `__mods` registry; `import {a, b as c} from './m.js'` becomes `const {a, b: c} = __mods.m`; `export const/function/ class` are rewritten to plain declarations and the factory returns the exported names. Order is fixed (data first, app last).

Two caveats worth knowing:

- the export-name scanner accepts `$` in identifiers (regex `[A-Za-z_$][\w$]*`) — names like `$` and `$$` from `render.js` must survive the rewrite or the bundle breaks at runtime (this bit v2; fixed);

- the injected preamble installs `window.onerror` and an `unhandledrejection` hook that write the failure into `document.title`, because `file://` pages show no console.

The build is rebuildable at any time (`python build_standalone.py`); `tests/` verify the bundle compiles (`node --check`), registers all 8 modules, keeps no ES-module imports and carries the per-language example labels.

8 Known environment quirks

- **Embedded-browser loopback latency.** Requests from a webview-style browser to 127.0.0.1 can take seconds even when `curl` answers in milliseconds. The client timeout is therefore 30 000 ms (`api.js`), and every button relabels to its in-flight state so slow requests are never silent.
- **Server warm-up is synchronous.** The first G2P call loads lexicons and rules (seconds on a cold start). `__main__` runs the warm-up *before* uvicorn starts; a thread-at-import version deadlocked uvicorn's event loop on per-module import locks (observed), so the `startup` event variant only covers `uvicorn server:app`, where imports are complete when the loop starts.
- **No-cache asset policy.** `/`, `*.js`, `*.css`, `*.html` are served with `Cache-Control: no-cache` — a stale cached `app.js` once hid new features entirely.
- **Port discipline.** The bridge binds `127.0.0.1:8421` only; the v2 workbench's 8420 is untouched and both can run side by side.
- **Shared origin state.** Settings live in `localStorage` (`covtl.settings.v3`); two consoles on the same origin and port share them — expected when v3 replaces v2 at the same port, surprising when testing both at once.

9 Extension guide

Add a language example. Append to `EXAMPLES` in `js/data.js`: {`label, text, mode, lang`}. The chip row, the standalone bundle and the tests' bundle check all derive from it (rebuild the standalone). Nothing server-side changes.

Add a panel. Follow the compiler pattern: a root element in `index.html` (or one your module creates), a class in its own `js/` file exporting the panel, wiring in `app.js`, CSS tokens from `css/app.css`. Add the module to `build_standalone.py`'s `ORDER` (dependency order, data first, app last).

Add an endpoint. Extend `server.py`: a Pydantic request model, the route, and structured errors via `api_error`/`guard_error`. Keep the v2 envelope. If the endpoint produces artifacts, register them in `job["paths"]` so `/api/job/{id}/files/{name}` serves them, and respect the `exports` flag for what is *offered* versus kept for the display.

The acceptance rule. Whatever you add, the brief's acceptance rule binds the display to the engine's answer: *block boundaries shown = the anchor list returned by the engine.* Never re-segment in the browser; if the engine's walk changes shape, let `StructureDrift` fail loudly rather than guessing.